

PyTorch, the parts you use

A tensor is an array that remembers what was done to it. That is the whole trick: it lets the framework work out the gradients you derived by hand in Session 8.

Tensors

<code>torch.tensor(x, dtype=torch.float32)</code>	Make a tensor. Models expect float32; a float64 array is a common first error.
<code>t.shape</code>	As in NumPy, and as in NumPy it is the first thing to print.
<code>t.view(-1, 1) / t.squeeze()</code>	Reshape. Most training bugs are a column vector meeting a flat one.
<code>t.detach().numpy()</code>	Back to NumPy, dropping the gradient history. Needed before plotting.

Gradients

<code>loss.backward()</code>	Work out the gradient of the loss for every parameter behind it.
<code>opt.step()</code>	Take one step downhill, using those gradients.
<code>opt.zero_grad()</code>	Clear them. PyTorch adds gradients up, so forgetting this silently corrupts training.
<code>with torch.no_grad():</code>	Turn the bookkeeping off. Use it for validation and prediction.

The loop, in the order it always goes

```
for epoch in range(n_epochs):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        opt.step()

    model.eval()
    with torch.no_grad():
```

`val_loss = loss_fn(model(x_val), y_val)`
Every line has a counterpart in the NumPy version you wrote by hand. If a line is unclear, go back to that one.

THE BUG EVERYONE WRITES ONCE

`model.train()` and `model.eval()` are not decoration.

Dropout and batch normalisation behave differently during training and during evaluation. Forget to switch and your validation number is measured on a different model than the one you will deploy.

Second most common: forgetting `opt.zero_grad()`, which quietly adds each batch's gradients to the last one's.

Making it train, and making it generalise

Nothing here is a recipe. Each technique treats a specific problem and carries a specific cost, so the question is always which problem you actually have.

Optimisers and the learning rate

`SGD(params, lr=...)`

The plain version: step directly downhill.

`SGD(..., momentum=0.9)`

Keeps some of the previous direction, which smooths a noisy descent.

`Adam(params, lr=...)`

Adapts the step size per parameter. The usual first choice.

`lr too high`

Loss jumps around, or becomes nan. Lower it by a factor of ten and try again.

`lr too low`

Loss falls, but far too slowly and smoothly. Raise it.

Reading a training run

Plot training and validation loss on the same axes, from the first epoch. Three shapes account for nearly everything.

Underfitting

Both losses high.

Both still falling together.

The model cannot represent the pattern.

Try: more capacity, better features, train longer.

Overfitting

Training loss keeps falling.

Validation loss turns upward.

The gap between them widens.

Try: stop earlier, reduce capacity, regularise.

Broken

Loss flat, or nan, or wildly erratic.

Not a tuning problem at all.

Check: learning rate, scaling, targets, shapes.

Overfit a tiny sample first: it should reach almost zero.

Regularisers: the problem each one treats

`early stopping`

Stop when validation stops improving. Cheapest of all, and the most under-used. Costs nothing but a validation split.

`weight_decay=...`

Penalises large weights. The same idea as Ridge in Block 2. Costs one value to tune.

`nn.Dropout(p=0.5)`

Randomly silences units while training, so no unit can be relied on alone. Costs slower convergence.

`nn.BatchNorm1d(n)`

Renormalises activations between layers. Often speeds up training. Interacts with dropout, and the reason it helps is still debated.

BEFORE YOU BELIEVE AN IMPROVEMENT

Run the same configuration with three different seeds.

A network trained twice with different random initialisation gives two different numbers. If your improvement is smaller than that variation, you have measured noise and not an effect.

Change one thing at a time, and decide your threshold for calling something an effect before you look at the result.